IoT course - Day 4 hands-on lab

Team lab guide

From a micro:bit to AWS IoT Core: gateway, certificates, shadow, rules, troubleshooting

Logtel, 2026 edition. Use it next to the Day 4 slides: they show the same steps.
Everywhere this guide says teamNN, use your own team: team01 to team10.

The chain you build today

```
micro:bit + IoT:bit --Wi-Fi, MQTT, port 1883, team user + password--> class gateway (Mosquitto)
class gateway --bridge, mutual TLS, the gateway's certificate--> AWS IoT Core (N. Virginia)
AWS IoT Core --rule--> DynamoDB table lab_telemetry (history)
AWS IoT Core --shadow delta, rule--> lab/teamNN/cmd --> gateway --> micro:bit (commands)
team app in CloudShell --mutual TLS, YOUR certificate--> AWS IoT Core   (Part 3)
Grafana Cloud class wall --mutual TLS, its own certificate--> AWS IoT Core   (Part 5)
```

Your team

What	Value
Your number	teamNN = team01 ... team10 (on your card)
Your topics	lab/teamNN/telemetry (readings, every 5 s) and lab/teamNN/cmd (commands)
Your thing and client ID	teamNN - the same name everywhere
On the table	micro:bit V2 on its IoT:bit board, USB cable, laptop (browser only)
On your card	MakeCode link; gateway address, user and password; AWS console link, user and password; the course page
On the whiteboard	Wi-Fi name and password (given on the day)
Course page	github.com/SADOT-SD/IOTlab - slides, this guide, the Part 3 commands to copy
AWS Region	N. Virginia (us-east-1) - check it top right in the console

Lab rules

- Use only your own team number: topics, client ID, thing.
- A private key stays in its file: no chat, no e-mail, no screenshots.
- Do not change or delete what another team created.
- Stuck for five minutes? Check the 'If it does not work' table, then call us.
- Everything is deleted after the course - certificates included.

Six parts, one chain

#	Part	Time	Done when ...
1	micro:bit to the gateway	60 min	your line moves on the class wall
2	AWS console tour	20 min	your messages scroll in the MQTT test client
3	Your own certificate	45 min	your team app prints your readings
4	Shadow and rules	40 min	a heart appears; DynamoDB keeps history
5	The class wall	15 min	ten teams on one Grafana dashboard
6	Break and fix	40 min	you name the cause from the logs

Do not move on until the check works - every part builds on the one before.

Part 1 - micro:bit to the class gateway (60 min)

Goal: your micro:bit sends its temperature and light level every 5 seconds, and reacts to commands. The micro:bit drives the IoT:bit's Wi-Fi chip over a serial link (pins P8 and P12) with text AT commands; the chip does Wi-Fi, TCP and MQTT. The class gateway checks your user and password and passes everything under lab/ on to AWS.

Steps

1. Open the MakeCode link on your card. If MakeCode shows a preview first, click Edit Code.
2. Change these values in the blocks (table below). Type them exactly as on your card and the whiteboard, with no extra spaces.
3. Connect the micro:bit with the USB cable and click Download. If MakeCode offers to connect the device, follow its steps; otherwise drag the downloaded .hex file onto the MICROBIT drive.
4. Power the IoT:bit from its own USB socket.
5. Wait up to about half a minute: a tick on the LEDs means the gateway accepted your login; a sad face means Wi-Fi or the gateway login failed.
6. Find your tile on the projected class wall.

Block	Placeholder	Change to
connect Wifi	`SSID = <Logtel Wi-Fi>`	only if it still says this: the Wi-Fi name (whiteboard)
connect Wifi	`KEY = <password>`	the Wi-Fi password on the whiteboard
Set MQTT client config	`clientID: teamNN`	your team, for example team07
Set MQTT client config	`username: teamNN`	your team (the same)
Set MQTT client config	`password: <on your card>`	your gateway password
publish (in 'every 5000 ms')	`lab/teamNN/telemetry`	for example lab/team07/telemetry
when Topic	`lab/teamNN/cmd`	for example lab/team07/cmd

Do not touch the backslashes. The message text is `{\"temp\":` + temperature + `\,\"light\":` + light level + `}`. The text travels inside an AT command, where a plain quote or comma would end it early; the Wi-Fi chip removes the backslashes and publishes `{"temp":23,"light":120}`.

Done when your temperature appears on the projected class wall.

If it does not work

What you see	Check
Nothing on the LEDs	Program downloaded? IoT:bit powered from its own USB socket?
Sad face	Wi-Fi name and password (whiteboard); gateway address, port, team user and password (card)
Sad face on every board	Gateway down, or the network blocks port 1883 - tell the lecturer
Tick, nothing on the wall	Topic spelling (lab/ in lowercase, your team); the backslashes
Restarts or freezes	Power the IoT:bit from its own USB socket
Another team's board drops out	Two boards logged in with the same team user

Same method as always: fix the lowest broken layer first.

Part 2 - find your data in AWS (20 min)

Goal: see your readings arrive in AWS IoT Core and send your first command. The MQTT test client in the console logs in with your console user (not with a certificate).

Steps

1. Open the AWS console link on your card and sign in with your team user and password.
2. Top right: choose the Region N. Virginia (us-east-1). Everything in the lab lives there.
3. Open IoT Core (search box at the top). Left menu: **Test > MQTT test client**.
4. Subscribe to a topic: Topic filter `lab/teamNN/#`, then Subscribe.
5. Watch your readings arrive every 5 seconds (the newest is on top).
6. Publish to a topic: Topic name `lab/teamNN/cmd`, message `{"led": "heart"}`, then Publish.

Done when your readings scroll in the test client and the heart appears on the micro:bit.

Your console user may read every lab/ topic (try `lab/#` to see the whole class) but may publish only under `lab/teamNN/`. The command travels test client -> AWS IoT Core -> bridge -> gateway -> micro:bit: the first end-to-end proof of the day.

If it does not work

What you see	Check
Nothing arrives	Region first (N. Virginia), then the topic (case-sensitive), then Part 1's tick
Publish refused or nothing happens	The topic must be your own: lab/teamNN/cmd
Heart does not appear	The 'when Topic' block must say lab/teamNN/cmd; press reset on the micro:bit

Wireshark: what the network still sees (5 min)

If Wireshark is installed on your laptop; otherwise watch the lecturer's screen.

1. Start a capture on the laptop's network card.
2. Reload the MQTT test client page.
3. Filter: `tls.handshake.type == 1`
4. Open a Client Hello and find the server name (SNI).
5. Everything after it is Application Data: encrypted.

The lesson: the network sees whom you talk to - not what you say. Reminder from Day 2: plain MQTT on port 1883 (our micro:bit to gateway leg) shows user, password and data.

Part 3 - your own certificate (45 min)

Goal: your team app gets its own identity and talks to AWS IoT Core directly with mutual TLS - no gateway in between. This is X.509 onboarding: the same steps as for a real device or a phone app.

Word	Meaning in this lab
Certificate (.cert.pem)	Your public ID card, signed by AWS. It may be shown to anyone.
Private key (.private.key)	The secret that proves the card is yours. It never leaves CloudShell.
Policy (lab-team-policy)	What the certificate may do: only lab/<its thing name>/...
Thing (teamNN)	Your device's name in AWS. The client ID must equal the thing name.
Amazon Root CA 1	Lets your app check that it really talks to AWS.

Steps

1. Open CloudShell: the >_ icon in the console's top bar (same Region). Wait for the prompt. One CloudShell tab per team.
2. Paste the first block - with your team number in the first line. Copy the blocks from the course page github.com/SADOT-SD/IOTlab: each block there has a copy button.
3. Paste the second block: attach the policy and the thing.
4. Paste the third block: download the CA, read the endpoint, list the files.
5. Paste the fourth block: it downloads team_app.py from the course page and runs it. Type heart and press Enter; then smile.
6. Try `team04 heart` (another team's topic). What happens, and why?
7. Stop the app with Ctrl+C.

Block 1 - create the key pair and the certificate (AWS signs it and marks it ACTIVE)

```
T=teamNN
ARN=$(aws iot create-keys-and-certificate --set-as-active \
  --certificate-pem-outfile $T.cert.pem --private-key-outfile $T.private.key \
  --query certificateArn --output text)
echo $ARN
```

Block 2 - give the certificate the team policy, and tie it to your thing

```
aws iot attach-policy --policy-name lab-team-policy --target $ARN
aws iot attach-thing-principal --thing-name $T --principal $ARN
```

Block 3 - the CA that lets you trust AWS, and your account's endpoint

```
curl -sO https://www.amazontrust.com/repository/AmazonRootCA1.pem
EP=$(aws iot describe-endpoint --endpoint-type iot:Data-ATS --output text)
echo $EP
ls
```

Block 4 - download the team app, install the MQTT library once, then run

```
R=https://raw.githubusercontent.com/SADOT-SD/IOTlab/main
curl -fsSLO $R/team_app.py
pip3 install --user paho-mqtt
python3 team_app.py $T $EP
```

Expected output (team03 shown):

```
connected to xxxx-ats.iot.us-east-1.amazonaws.com:8883 as team03
lab/team03/telemetry  {"temp": 24, "light": 131}
heart
sent to lab/team03/cmd: heart
```

Check your identity

```
openssl x509 -in $T.cert.pem -noout -subject -issuer -dates
```

Expected: subject 'AWS IoT Certificate', valid until 31 December 2049. Read the issuer line: who signed your certificate? (If CloudShell says openssl is not found: `sudo dnf install -y openssl`.)

Where in the console	What you should see
Security > Certificates	your certificate is ACTIVE
All devices > Things > teamNN > Certificates	it is attached to your thing
The certificate's page > Policies	lab-team-policy is attached

Done when your team app prints your readings, 'heart' changes the LEDs, and all three console checks are green - you onboarded a client with X.509.

If it does not work

What you see	Check
AccessDenied or 'not found' in CloudShell	Region N. Virginia? T set to your own team?
certificate verify failed	AmazonRootCA1.pem missing or in another folder (run ls)
Closed right after connecting	Policy or thing not attached (block 2), or T is another team
Two copies keep kicking each other	Only one copy of the app per team (one client ID)
CloudShell restarted (20-30 min idle)	Run T=teamNN and the EP= line again - the files stay
team_app.py: No such file	Run the R= and curl lines again; or download team_app.py from the course page and use CloudShell Actions > Upload file
Cannot connect at all (port 8883 blocked)	python3 team_app.py $T $EP 443

Remember: the .private.key file is your team's identity. Never copy it out of CloudShell, never paste it into a chat.

Part 4 - shadow and rules (40 min)

Goal: send a command through the device shadow, and find your history in DynamoDB. The shadow keeps the desired state even when the device is offline.

4a. Commands through the shadow

1. IoT Core > All devices > Things > teamNN > Device Shadows.
2. Classic Shadow > Edit.
3. Set desired.led (below) and press Update.
4. Watch the micro:bit.
5. Try smile - then hi.

```
{
  "state": {
    "desired": {"led": "heart"}
  }
}
```

Done when the micro:bit changes within a few seconds of Update.

The path: console -> shadow -> delta -> rule -> lab/teamNN/cmd -> gateway -> micro:bit. The micro:bit never reports back, so every new desired value makes a new delta message.

4b. The two lab rules (ready-made, read only)

IoT Core > Message routing > Rules. Open each rule and read its SQL:

Rule	SQL	Action
lab_telemetry_to_dynamodb	`SELECT temp, light, topic(2) AS team, timestamp() AS ts FROM 'lab/+/telemetry'`	DynamoDB table lab_telemetry
lab_shadow_to_cmd	`SELECT state.led AS led FROM '$aws/things/+/shadow/update/delta'`	Republish to lab/${topic(3)}/cmd

topic(2) of lab/team03/telemetry is team03; topic(3) of $aws/things/team03/shadow/update/delta is team03.

4c. History in DynamoDB

1. DynamoDB > Tables > lab_telemetry.
2. Explore items.
3. Query: team = teamNN.
4. Sort by ts, newest first.

Done when you find your own rows - and they keep growing.

ts is milliseconds since 1970: the rule adds the time, the micro:bit does not need a clock. The class wall forgets when it is closed - the table remembers.

Part 5 - the class wall (15 min, lecturer demo)

- A third identity: the dashboard has its own certificate (client ID grafana-wall).
- Its policy may only read lab/<team>/telemetry topics - it cannot send commands.
- One tile per team; a tile turns orange at 30 degrees C (warm your micro:bit with your hand).
- Live only: Grafana's MQTT data source stores nothing and cannot alert - history is in DynamoDB.

Part 6 - break and fix (40 min)

The lecturer breaks one thing for your team, or you draw a card. For each challenge write down: which layer, what evidence, what fix - then tell the class.

#	Challenge	What you see
1	Sad face	The micro:bit never reaches the gateway any more
2	Cut off	The team app connects and is dropped at once
3	One-way	Readings arrive, but commands never reach the board
4	Tug of war	Two apps keep disconnecting each other
5	Refused	The team app fails after a 'small improvement'
6	Invisible	Your line is missing from the class wall

Find the broken layer - bottom up

Layer	Ask	Look at
Power, board	Is it on? Is our program running?	LEDs, MakeCode
Wi-Fi (L1-2)	Did the IoT:bit join the network?	tick or sad face on the micro:bit
IP, DNS (L3)	Address? Does the name resolve?	nslookup
TCP port (L4)	Can we reach the port at all?	nc -zv (CloudShell)
TLS (L5-6)	Do the certificates check out?	openssl, the team app's error
MQTT (L7)	Connected? Right topic? Right ID?	MQTT test client
Policy (L7)	Is it allowed?	AWSIotLogsV2

Where to look

- Who connected and who was thrown out: subscribe to `$aws/events/presence/#` in the MQTT test client (field disconnectReason).
- Why AWS said no: CloudWatch > Log groups > `AWSIotLogsV2` > Logs Insights, with this query (your team):

```
fields @timestamp, eventType, status, reason
| filter clientId = "teamNN" | sort @timestamp desc
```

- Certificate state: Security > Certificates (ACTIVE / INACTIVE).
- The micro:bit itself never appears in the AWS logs: it talks to the class gateway, whose log the lecturer can show.

Appendix - symptom, cause, fix

Symptom	Likely cause	Fix
micro:bit never joins Wi-Fi	wrong name or password, 5 GHz network	whiteboard password; 2.4 GHz only
Wall shows nothing for us	topic typo (Lab/ vs lab/)	copy the topic from the card
'certificate verify failed'	wrong or missing CA file	AmazonRootCA1.pem, right folder
Closed right after connecting	policy, inactive certificate	AWSIotLogsV2, certificate state

Symptom	Likely cause	Fix
Two apps keep kicking each other	same client ID	one ID per running app
Publish with QoS 2 refused	AWS IoT has no QoS 2	use QoS 0 or 1

Appendix - words used today

Word	Meaning
MQTT	Publish/subscribe messaging: devices publish to a topic, the broker delivers to subscribers
Broker	The MQTT server: our class gateway (Mosquitto) and AWS IoT Core
Topic	The address of a message, e.g. lab/team03/telemetry; + and # are wildcards when subscribing
QoS	Delivery level: 0 = at most once, 1 = at least once (AWS IoT Core has no QoS 2)
Bridge	The gateway's own connection to AWS IoT Core that carries lab/ topics both ways
Thing	A device's entry in the AWS IoT registry; its name is also the client ID
Certificate / private key	Public ID card signed by a CA / the secret that proves it is yours
Mutual TLS (mTLS)	Both sides show a certificate: the device checks AWS, AWS checks the device
Policy	The list of what a certificate may do (connect, publish, subscribe, receive)
Shadow	A JSON document per thing with desired and reported state; delta = the difference
Rule	SQL on incoming messages that triggers actions (store in DynamoDB, republish)
Endpoint	Your account's AWS IoT address: xxxx-ats.iot.us-east-1.amazonaws.com